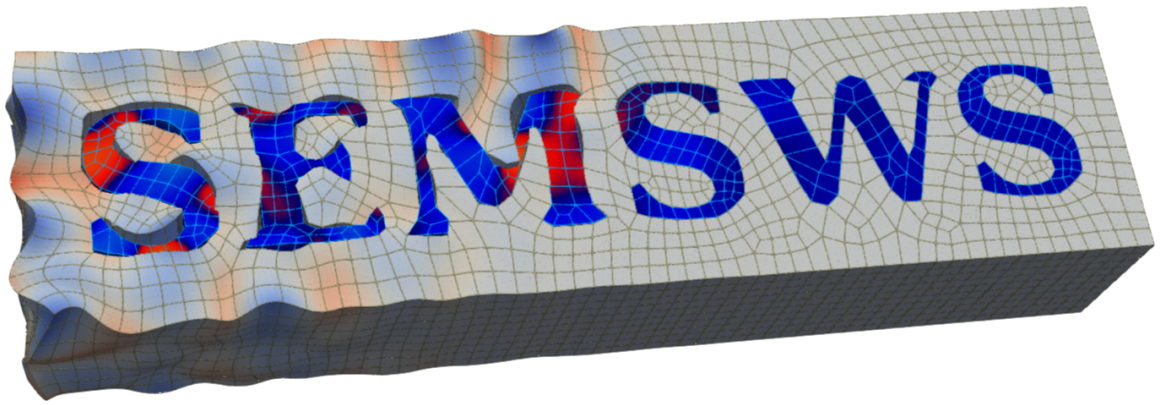

Spectral Element Method
Seismic Wave Simulator

User Manual

Version 0.1

May 12, 2026

This manual was translated from the original Japanese version with the assistance of AI (Claude, Anthropic). While effort has been made to ensure accuracy, some expressions may differ from the original. For the most up-to-date and authoritative version, please refer to the Japanese edition.

Contents

Chapter 1

Introduction

1.1 What is SEMSWS?

SEMSWS (Spectral Element Method – Seismic Wave Simulator) is a seismic wave propagation simulation solver based on the spectral element method (SEM). It is built on the finite element library MFEM[1]. MFEM is an open-source library developed primarily at Lawrence Livermore National Laboratory. Many of the core capabilities of SEMSWS—high-order finite element spaces, support for diverse mesh formats, MPI-parallel assembly, and GPU/CPU portability via CUDA/HIP/OpenMP backends—are underpinned by the MFEM framework. We gratefully acknowledge the MFEM development team. This enables execution on both CPUs and GPUs (CUDA/HIP) from a single source code.

SEM, a widely used numerical method for solving seismic wave equations, employs high-order Lagrange interpolation based on Gauss–Lobatto–Legendre (GLL) points. Its key advantages include efficient explicit time integration through mass matrix diagonalization, support for unstructured meshes, natural incorporation of free-surface boundary conditions via the weak formulation, and high-accuracy, low-dispersion solutions based on spectral convergence.

1.2 Supported Physics Models

SEMSWS supports the following wave equations in two and three dimensions:

- **Isotropic Elastic Wave Equation** (Isotropic Elastic)
- **Isotropic Acoustic Wave Equation** (Isotropic Acoustic)
- **Viscoelastic / Viscoacoustic Attenuation** (Viscoelastic / Viscoacoustic)

Future updates are planned to add support for VTI, full anisotropy, and solid–fluid coupled domains.

The polynomial order can be set to any value; however, the GPU implementation imposes an upper limit due to shared memory constraints of the stiffness kernel.

Time integration uses the explicit Newmark scheme ($\beta = 0$, $\gamma = 1/2$, equivalent to the central difference method). This scheme is second-order accurate in time and conditionally stable under the CFL condition.

For absorbing boundary conditions, a sponge absorbing layer (Cerjan damping) is implemented. Perfectly matched layers (PML) are planned for future development.

Full Waveform Inversion (FWI) functionality is currently under development. It is partially supported only for 2D acoustic cases, and testing remains insufficient.

[1] https://mfem.org/

1.3 Key Aspects of the Spectral Element Method

The main SEM parameter that users specify in the configuration file is the **polynomial order** (`order`).

When `order` = N, $N + 1$ GLL points are placed along each edge of every element. For seismic wave simulations, `order` = 4–5 is typical.

The essential advantage of SEM is that using GLL points for both interpolation and quadrature yields a diagonal mass matrix. This allows the acceleration field to be computed by simple element-wise division, eliminating the need to solve a system of equations. The stiffness term **KU** is not assembled as a matrix but computed matrix-free through element-level operations, reducing both memory usage and computational cost.

1.4 Structure of This Manual

Chapter 2: Installation Describes the procedure from preparing the dependencies (hypre, MFEM) through the build process. Includes build configuration examples for various computing environments (local, ES, LUMI).

Chapter 3: Quick Start Walks through the complete workflow—from writing a configuration file to running a simulation and visualizing the results—using a 2D acoustic wave simulation as an example.

Chapter 4: Configuration File Reference Provides a comprehensive description of all options in config.yaml.

Chapter 5: Tools and Scripts Explains how to use the mesh partitioning, refinement, and quality evaluation tools, as well as visualization scripts.

Chapter 6: Application Examples Presents example cases including analytical solution benchmarks and 3D viscoelastic models.

Chapter 7: Performance Shows scaling results on ES and LUMI, along with GPU/CPU performance comparisons.

Chapter 2

Installation

2.1 Requirements

The following software is required to build SEMSWS.

2.1.1 Required

Table 2.1: Required dependencies

Software	Notes
C++17 compiler	`-std=c++17` support
CMake	Build system
MPI	Parallel computing
MFEM	Must be built with MPI enabled
ADIOS2	Parallel I/O (BP format)
HDF5	Waveform output
yaml-cpp	Automatically fetched by CMake (no manual installation needed)

2.1.2 Optional

Table 2.2: Optional dependencies

Software	Notes
CUDA	NVIDIA GPU support
HIP	AMD GPU support

2.2 Building the Dependencies

SEMSWS is built on top of the MFEM library, which in turn depends on hypre. These must be built with consistent precision settings and GPU backends.

2.2.1 Building hypre

CPU version

```
cd hypre/src
./configure --enable-single --with-MPI
make -j$(nproc)
```

For double precision, omit `--enable-single`.

CUDA version (NVIDIA GPU)

```
cd hypre/src
./configure \
  --enable-single --enable-mixedint \
  --enable-unified-memory \
  --with-cuda --enable-gpu-aware-mpi \
  CUDA_HOME=/path/to/cuda HYPRE_CUDA_SM=80
make -j$(nproc)
```

HIP version (AMD GPU)

```
cd hypre/src
./configure \
  --enable-single --enable-mixedint \
  --enable-unified-memory \
  --with-hip --enable-gpu-aware-mpi \
  CC=hipcc CXX=hipcc
make -j$(nproc)
```

–enable-unified-memory is likely recommended for GPU environments.

2.2.2 Building MFEM

MFEM is the core finite element library for SEMSWS. Build it with CMake using the following options.

Build example (CPU, single precision)

```
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=mpicxx \
  -DMFEM_USE_MPI=ON \
  -DMFEM_USE_HDF5=ON \
  -DHDF5_DIR=/path/to/hdf5 \
  -DMFEM_USE_NETCDF=ON \
  -DMFEM_PRECISION=single \
  -DHYPRE_DIR=/path/to/hypre/src/hypre
cmake --build build -j$(nproc)
```

NETCDF is not strictly required, but it is needed for reading meshes generated by CUBIT or similar tools.

Precision consistency The precision settings of hypre (`--enable-single`) and MFEM (`MFEM_PRECISION`) must match. Since SEMSWS automatically inherits MFEM's precision setting, no additional configuration is needed on the SEMSWS side as long as hypre and MFEM are consistent.

For further details on building MFEM, refer to the official documentation[1].

2.3 Building SEMSWS

2.3.1 Basic Build Procedure

Run the following commands from the root directory of the source code:

```
# 1. CMake configure
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
```

[1] https://mfem.org/building/

```
  -DMFEM_DIR=/path/to/mfem/build

# 2. Build
cmake --build build -j$(nproc)
```

Set `MFEM_DIR` to the path of the MFEM build directory or install directory.

2.3.2 CMake Options

Table 2.3: CMake options

Option	Default	Description
`MFEM_DIR`	—	Path to MFEM (required)
`CMAKE_BUILD_TYPE`	—	`Release`/`Debug`
`CMAKE_CXX_COMPILER`	Auto-detected	MPI compiler
`CMAKE_CUDA_ARCHITECTURES`	`70;80`	CUDA architectures
`AMDGPU_TARGETS`	`gfx90a`	AMD GPU architectures

2.3.3 GPU Build

If MFEM was built with CUDA or HIP support, SEMSWS will automatically be built as a GPU-enabled binary.

2.4 Build Configuration Examples

The following sections provide build procedures for various environments.

2.4.1 Local Environment (WSL2/Ubuntu, CPU only)

Example configuration for single precision, CPU only.

```
# hypre
cd $HOME/program_ubuntu/hypre/src
./configure --enable-single --with-MPI
make -j$(nproc)

# MFEM
cd $HOME/program_ubuntu/mfem
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=mpicxx \
  -DMFEM_USE_MPI=ON \
  -DMFEM_USE_HDF5=ON -DMFEM_USE_NETCDF=ON \
  -DMFEM_PRECISION=single \
  -DHYPRE_DIR=$HOME/program_ubuntu/hypre/src/hypre
cmake --build build -j$(nproc)

# SEMSWS
cd $HOME/program_ubuntu/SEMSWS
cmake -B build \
  -DCMAKE_BUILD_TYPE=Debug \
  -DMFEM_DIR=$HOME/program_ubuntu/mfem/build
cmake --build build -j$(nproc)
```

2.4.2 Earth Simulator (NVIDIA A100 GPU)

Example for the GPU partition (NVIDIA A100) of JAMSTEC's Earth Simulator. The compiler used is NVIDIA HPC SDK 24.7 (nvc++/nvcc). The following configuration has been verified as of April 2025: NVIDIA HPC SDK 24.7, CUDA 12.0.0, hypre 2.31.0, METIS 5.1.0, HDF5 1.10.5.

```
# Load modules
module load HDF5/1.10.5
module load NetCDF4/all
module load NVIDIAHPCSDK/24.7/nvhpc-hpcx-cuda12
module load METIS/5.1.0
module load CUDA/12.0.0

# hypre
cd $HOME/program_gpu/hypre/src
./configure \
  --enable-single --enable-mixedint \
  --enable-unified-memory \
  --with-cuda --enable-gpu-aware-mpi \
  CUDA_HOME=/opt/share/CUDA/12.0.0 HYPRE_CUDA_SM=80
make -j$(nproc)

# MFEM (depends on hypre above)
cd $HOME/program_gpu/mfem
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=nvc++ \
  -DCMAKE_CUDA_COMPILER=nvcc \
  -DCMAKE_CUDA_ARCHITECTURES=80 \
  -DMFEM_USE_MPI=ON \
  -DMFEM_USE_CUDA=ON \
  -DMFEM_USE_HDF5=ON \
  -DMFEM_USE_NETCDF=ON \
  -DMFEM_PRECISION=single \
  -DHYPRE_DIR=$HOME/program_gpu/hypre/src/hypre \
  -DMETIS_DIR=/opt/share/METIS/5.1.0
cmake --build build -j$(nproc)

# SEMSWS
cd $HOME/program_gpu/SEMSWS
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=nvc++ \
  -DCMAKE_CUDA_COMPILER=nvcc \
  -DCMAKE_CUDA_ARCHITECTURES=80 \
  -DCMAKE_CXX_STANDARD=17 \
  -DMFEM_DIR=$HOME/program_gpu/mfem/build \
  -Dadios2_DIR=$HOME/program_cpu/ADIOS2/build
cmake --build build -j
```

SEMSWS only consumes MFEM (via `MFEM_DIR`) and ADIOS2 (via `adios2_DIR`); hypre and METIS are linked transitively through MFEM, so they do not appear on the SEMSWS cmake line.

2.4.3 LUMI (AMD MI250X GPU)

Example for CSC Finland's LUMI supercomputer (LUMI-G partition). The compiler is ROCm 6.4.4's `hipcc`, and the MPI implementation is Cray MPICH. The following configuration has been verified as of April 2026: ROCm 6.4.4, Cray PE 25.09 (CCE 20.0.0), hypre 3.1.0, METIS 5.1.0 (cpeCray-25.09-idx32-fp32), cray-hdf5 1.14.3.7.

```
# Load modules
module load LUMI/25.09 partition/G
module load METIS/5.1.0-cpeCray-25.09-idx32-fp32
module load cray-hdf5/1.14.3.7

# hypre
cd $FD/program/hypre/src
./configure \
  --enable-single --enable-mixedint \
  --enable-unified-memory \
  --with-hip --enable-gpu-aware-mpi \
  CC=hipcc CXX=hipcc \
  LDFLAGS="-L/opt/cray/pe/mpich/9.0.1/gtl/lib" \
  LIBS="-lmpi_gtl_hsa"
make -j$(nproc)

# MFEM (depends on hypre above)
cd $FD/program/mfem
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=hipcc \
  -DMFEM_USE_MPI=ON \
  -DMFEM_USE_HIP=ON \
  -DAMDGPU_TARGETS=gfx90a \
  -DMFEM_USE_HDF5=ON \
  -DMFEM_USE_NETCDF=ON \
  -DMFEM_PRECISION=single \
  -DHYPRE_DIR=$FD/program/hypre/src/hypre \
  -DHDF5_INCLUDE_DIRS=/opt/cray/pe/hdf5/1.14.3.7/cray/20.0/include \
  -DHDF5_LIBRARIES="/opt/cray/pe/hdf5/1.14.3.7/cray/20.0/lib/libhdf5_cpp.so
     ;\
/opt/cray/pe/hdf5/1.14.3.7/cray/20.0/lib/libhdf5.so"
cmake --build build -j$(nproc)

# SEMSWS
cd $FD/program/SEMSWS
cmake -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=hipcc \
  -DAMDGPU_TARGETS=gfx90a \
  -DMFEM_DIR=$FD/program/mfem/build \
  -Dadios2_DIR=$FD/program/ADIOS2/build
cmake --build build -j
```

SEMSWS only consumes MFEM (via `MFEM_DIR`) and ADIOS2 (via `adios2_DIR`); hypre, METIS and the HDF5 locations are linked transitively through MFEM, so they do not appear on the SEMSWS cmake line. Use `srun` to launch MPI jobs.

2.5 Verifying the Build

Upon successful build, executables are generated under `build/src/`.

Table 2.4: Generated executables

Executable	Description
`semsws`	Main simulation program
`partition_mesh`	Mesh pre-partitioning
`refine_mesh`	Mesh refinement
`evaluate_mesh`	Mesh quality evaluation
`combine_mesh`	Combining partitioned meshes (for visualization)
`bp2gf`	Convert ADIOS2 BP to GLVis format

To verify that the build completed successfully:

```
./build/src/semsws --help
```

Verification through tests is also possible. Running the tests requires Python 3.9 or later, along with `pytest` and `numpy`.

```
# Run all tests (default is CPU, serial run)
pytest test/ --build-dir ./build -v

# MPI parallel with GPU device
pytest test/ --build-dir ./build --np 4 --device cuda -v

# Waveform verification tests only
pytest test/waveform/ --build-dir ./build -v

# Output format consistency tests only
pytest test/consistency/ --build-dir ./build -v
```

The available tests fall into the following two categories.

Waveform verification tests (`test/waveform/`) These tests compare simulation results against reference waveforms and verify that the normalized L2 norm falls within a specified threshold. Cases whose names contain `analytic` are compared against analytical solutions; all other cases are compared against SPECFEM. The following cases are included:

Table 2.5: Waveform verification test cases

Dimension	Physics model
2D	acoustic, elastic, acoustic_visco, elastic_visco
3D	acoustic_analytic, elastic, elastic_analytic, elastic_visco

Passing the `--keep-results` option preserves the simulation outputs under `results/` inside each test case directory (e.g. `test/waveform/3D/elastic/`), so that they can be inspected file by file against the reference waveforms in the adjacent `ref/`:

```
pytest test/waveform/ --build-dir ./build --keep-results -v
```

Output format consistency tests (`test/consistency/`) These tests verify that the receiver waveforms are numerically identical across different output formats (ASCII, HDF5, SU).

2.6 Installing GLVis (Recommended)

GLVis is a lightweight visualization tool published by the MFEM project.[2] It renders the native MFEM mesh / grid-function files that SEMSWS writes under `results/wavefield/` and lets you watch the wavefield evolve frame-by-frame during or after a run. It is not strictly required to use SEMSWS, but having it installed makes the Quick Start tutorial in Chapter 3 much easier to follow — you can sanity check a simulation with one command instead of having to write a plot script first.

2.6.1 Build

GLVis links against the same MFEM install you built in the previous section; no separate MFEM build is needed.

```
cd ~/program_ubuntu
git clone https://github.com/GLVis/glvis.git
cd glvis
make -j MFEM_DIR=../mfem
```

The resulting executable is at `glvis` in the checkout. Put it on your `PATH` or keep the full path handy.

System packages for SDL / OpenGL / fontconfig / freetype must be available; on Debian / Ubuntu:

```
sudo apt install libsdl2-dev libglew-dev libfontconfig-dev \
                 libfreetype-dev
```

2.6.2 Basic Usage

Launch the viewer on a mesh file alone, or on a mesh + grid-function pair written by SEMSWS:

```
# Mesh only
glvis -m mesh.mesh

# Mesh + a single wavefield snapshot
glvis -m results/wavefield/mesh.mesh -g results/wavefield/sol_000500.gf

# Partitioned (MPI-split) mesh - N = number of ranks that wrote it
glvis -np 2 -m results/wavefield/mesh -g results/wavefield/sol_000500
```

For the partitioned form, GLVis loads `mesh.000000`, `mesh.000001`, …and the matching `sol_000500.000000` … produced by an MPI run and stitches them back into a single view.

See the GLVis README[3] for the full set of options and the in-window key bindings.

[2] https://glvis.org/
[3] https://github.com/GLVis/glvis

Chapter 3

Quick Start

This chapter walks through the complete workflow—from writing a configuration file to running a simulation and visualizing the results—using a 2D acoustic wave simulation as an example. The example used here is included in `examples/visualization/2D/acoustic/`.

3.1 Configuration File

In SEMSWS, all simulation parameters are specified in a single YAML file (`config.yaml`). The following sections explain the configuration file for this example.

3.1.1 Basic Simulation Settings

```yaml
simulation:
  dimension: 2
  order: 4

  time:
    steps: 10000
    dt: 0.0001
```

`dimension` specifies the spatial dimension (2 or 3), and `order` specifies the polynomial order (the order of the spectral element method). The `time` section sets the number of time steps and the time step size Δt. In this example, $\Delta t = 0.0001\,\mathrm{s}$ with 10000 steps, giving a total simulation time of 1 second.

3.1.2 Mesh

```yaml
mesh:
  type: internal
  origin: [0.0, 0.0]
  size: [1000.0, 500.0]
  elements: [50, 25]
  save: true
  max_freq: 75.0
  ppw: 5.0
```

`type: internal` instructs SEMSWS to automatically generate a regular/structured mesh internally. `size` specifies the domain size (in meters), and `elements` specifies the number of elements. In this example, a $1000\,\mathrm{m} \times 500\,\mathrm{m}$ domain is divided into 50×25 elements. When `save` is specified, a GLVis-compatible mesh is saved to the results directory. `max_freq` and `ppw` are parameters for mesh quality evaluation, representing the maximum frequency and the

points per wavelength, respectively. They are used in the summary.txt output and by the evaluate_mesh program described later.

To use an external mesh (GMSH, CUBIT/ExodusII, etc.), set `type: external` and specify the file path with `file:`.

3.1.3 Material Properties

```
material:
  type: isotropic_acoustic
  format: constant
  vp: 1500.0
  rho: 1000.0
  attenuation:
    enabled: false
```

This defines a homogeneous acoustic medium. `type` selects the physics model (`isotropic_acoustic`), and `format: constant` indicates a homogeneous model. The P-wave velocity is set to 1500 m/s and the density to 1000 kg/m^3.

3.1.4 Boundary Conditions

```
boundary:
  absorbing:
    type: cerjan
    sides: []
    thickness: 100.0
    alpha: 0.4

  dirichlet:
    attributes: [top, right, left, bottom]
```

`dirichlet` applies Dirichlet (fixed) boundary conditions on all sides. The absorbing boundary (Cerjan sponge layer) is disabled by setting `sides: []`. To enable absorbing boundaries, specify the sides to apply them to, e.g., `sides: [top, right, left, bottom]`. For marine reflection or refraction seismic survey simulations, **the sea surface must be set as a Dirichlet boundary** (`dirichlet: [top]`).

3.1.5 Sources

```
sources:
  list:
    - id: 1
      name: "Source 1"
      type: pressure
      location: [500.0, 400.0]
      wavelet:
        type: ricker
        frequency: 30.0
        amplitude: 1.0e10
        delay: 0.05
```

A single pressure point source with a 30 Hz Ricker wavelet is placed near the center of the domain (500 m, 400 m). `delay` is the time (in seconds) at which the wavelet reaches its peak.

3.1.6 Receivers

```
receivers:
  output:
    format: ascii
  type: [PS]

  list:
    - name: "R001"
      location: [400.0, 300.0]
    - name: "R002"
      location: [500.0, 300.0]
    - name: "R003"
      location: [600.0, 300.0]
```

Three receivers are placed to record the pressure scalar field (`PS`). The output format is `ascii`, producing text files with the first column as time and the second column as amplitude. Other available `format` options include `hdf5` and `su`. `su` is the Seismic Unix format.

3.1.7 Wavefield Output

```
simulation:
  output:
    wavefield:
      enabled: true
      interval: 1000
      fields: ["PS"]
      formats:
        - type: "glvis"
        - type: "paraview"
          refinement: 4
          data_format: "binary32"
        - type: "gmt"
          resolution: [100, 50]
```

Wavefield snapshots are output every 1000 steps. Up to three output formats can be specified simultaneously:

- **GLVis** — MFEM native format. Interactively visualized using the `glvis` command
- **ParaView** — VTU format. Suitable for 3D visualization with ParaView
- **GMT** — Text grid format (`x y value`). Grid resolution is specified with `resolution`

3.1.8 Device Settings

```
device:
  type: cpu
```

Choose from `cpu`, `cuda`, or `hip`. Even with a GPU build, specifying `cpu` will run on the CPU (though it will be significantly slower; a CPU build should be used for CPU execution).

3.2 Running the Simulation

Run the following command in the directory containing the configuration file:

```
cd examples/visualization/2D/acoustic

# Run with 4 processes
mpirun -np 4 ../../../../build/src/semsws \
  --config config.yaml
```

You can also use the included `run.sh`:

```
bash run.sh 4    # argument is the number of MPI processes
```

Upon completion, the following are generated in the `results/` directory:

Table 3.1: Output files

Path	Description
`summary.txt`	Simulation information (material properties, mesh, performance, etc.)
`R001_0001.p` etc.	Receiver waveforms (ASCII text)
`wavefield/gmt/`	GMT snapshots
`wavefield/paraview/`	ParaView VTU files
`wavefield/glvis/`	GLVis binary files
`material/`	Material property distribution output
`mesh.mesh`	The mesh used

3.3 Examining the Results

3.3.1 Summary

The file `results/summary.txt` contains a summary of the simulation. Below is an example output from this tutorial:

```
Discretization:
  Order: 4
  Elements: 1250
  DOFs: 20301

Time stepping:
  dt: 0.0001 s
  nt: 10000
  Simulation time: 1 s

Numerical stability:
  h_min: 20.00 m, h_max: 20.00 m
  CFL dt_max: 7.3535e-04 s (dt/dt_max = 0.14)
  PPW: 12.5 (f=30.0 Hz)

Performance:
  Setup time:  0.18 s
  Run time:    2.27 s
  Total time:  2.44 s
  DOFs/sec:    8.96e+07
```

In particular, verify that `CFL dt_max` is greater than the specified Δt. If $\Delta t > \Delta t_{\max}$, the computation becomes unstable and diverges. `PPW` (Points Per Wavelength) is the number of grid points per wavelength; a value of 5 or more is a common guideline.

3.3.2 Wavefield Snapshots

The wavefield can be visualized using the included Python script:

```
python3 scripts/plot/plot_wavefield_2d.py --results-dir ./results
```

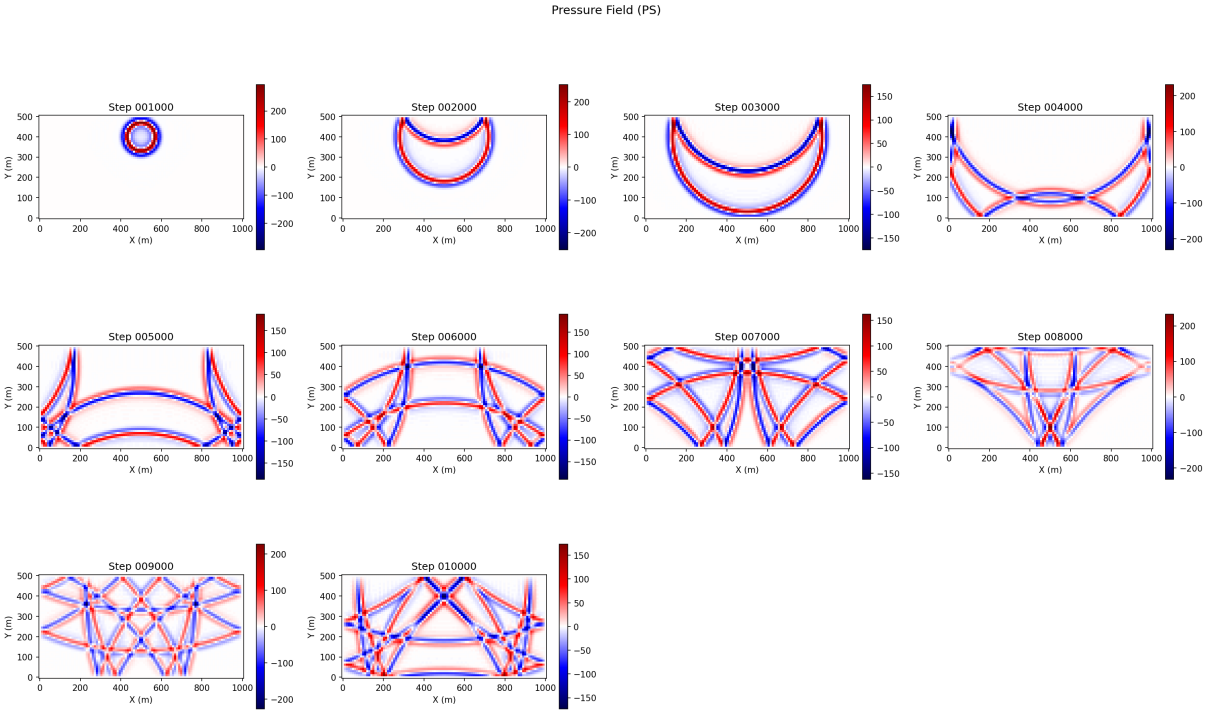

Figure 3.1: Pressure field snapshots (at 1000-step intervals).

In this example, Dirichlet boundary conditions are applied on all sides, so wave reflections at the boundaries can be observed. In actual seismic wave simulations, absorbing boundary conditions are used to suppress artificial reflections.

3.3.3 Receiver Waveforms

The receiver waveform files are in two-column ASCII text format. The first column is time (seconds) and the second column is amplitude (pressure). They can be easily plotted with gnuplot, Python, or similar tools.

Chapter 4

Configuration File Reference

All parameters for a SEMSWS simulation are specified in a single YAML file (`config.yaml`). This chapter provides a comprehensive description of all configuration options. For basic usage, refer to Chapter 3.

4.1 YAML Basics

The configuration file is written in YAML (YAML Ain't Markup Language) format. The following summarizes the basic syntax rules of YAML.

- Hierarchy is expressed through indentation with spaces.
- Key–value pairs are written in the form `key: value`
- Lists are denoted with `-`
- Everything after `#` is treated as a comment
- Strings can be written without quotation marks, but those containing special characters should be enclosed in `"..."`

```yaml
# Scalar values
name: "My Simulation"
order: 4
dt: 0.0001

# Nested structure
simulation:
  dimension: 2
  time:
    steps: 10000
    dt: 0.0001

# List
sides: [bottom, right, top, left]

# List (multi-line format)
list:
  - name: "R001"
    location: [400.0, 300.0]
  - name: "R002"
    location: [500.0, 300.0]
```

4.2 Strict-Key Validation

The YAML loader validates every block against a known-key set. An unknown sibling key anywhere in the document — a typo, a renamed or removed legacy key, or a sub-section copied from an older config — is a fatal error. Instead of silently ignoring the stray key, the loader aborts with a message of the form:

```
Error validating config: Unknown key `<bad_key>` in <section>
  (allowed: <comma-separated list of valid keys for that block>)
```

The same mechanism also rejects a handful of removed legacy forms with tailored migration hints:

- `boundary.free_surface` was removed. Elastic free-surface is the natural boundary condition (delete the block); acoustic free-surface (pressure = 0) is now declared via `boundary.dirichlet.attr`
- `simulation.output.wavefield.format` and `receivers.output.format` (singular) are replaced by `formats:` (plural list of mapping entries) — see §4.5.5 / §4.10.1.
- `component: <int>` (singular) inside an output-format entry is replaced by `components: [<int>, ...]`.
- `simulation.checkpoint_storage: host_memory` is renamed to `memory`.
- `material.format: ascii` is renamed to `grid`.

When porting an older configuration file, apply the rename(s) above and delete any keys that are no longer listed in this chapter.

4.3 Overall Structure of the Configuration File

The overall structure of the configuration file is as follows:

```yaml
name: "..."                # Simulation name (optional)
description: "..."         # Description (optional)
simulation:                # Basic simulation settings and output
mesh:                      # Mesh
material:                  # Material properties
boundary:                  # Boundary conditions
sources:                   # Sources
receivers:                 # Receivers
device:                    # Execution device
```

4.4 Root-Level Parameters

Key	Type	Required	Default	Description
`name`	string	Optional	`"Seismic wave simulation"`	Simulation name. Recorded in `summary.txt`
`description`	string	Optional	`"No description provided."`	Free-form description

4.5 simulation Section

4.5.1 Basic Parameters

Key	Type	Required	Values	Description
`dimension`	int	Required	2, 3	Spatial dimension
`order`	int	Required	≥ 1	Polynomial order (number of GLL points = $N + 1$)

The GPU implementation imposes an upper limit on `order` due to shared memory constraints.

4.5.2 Time Integration (time)

Key	Type	Required	Default	Description
`time.dt`	real	Required	—	Time step size (s)
`time.steps`	int	Required	—	Number of time steps
`time.cfl_factor`	real	Required	—	CFL safety factor (0–0.7)
`time.t0`	real	Optional	0.0	Simulation start time (s)

Δt must satisfy the CFL condition ($\Delta t < \Delta t_{\max}$). `cfl_factor` is used for mesh quality evaluation. The total simulation time is determined by `steps` $\times \Delta t$.

4.5.3 Simulation Mode

Key	Type	Required	Default	Description
`mode`	string	Optional	`forward`	`forward`, `inversion`, `misfit_only`
`misfit_type`	string	Optional	`l2_waveform`	`l2_waveform`, `normalized_correlation`
`num_checkpoints`	int	Optional	10	Number of Revolve checkpoints
`checkpoint_storage`	string	Optional	`"memory"`	`"disk"`, `"memory"`
`checkpoint_device`	string	Optional	`"auto"`	`"host"`, `"device"`, `"auto"`
`checkpoint_dir`	string	Conditional	`"./checkpoints/"`	Required when using `disk` (not yet supported)
`kernel_output_dir`	string	Optional	`"./kernels/"`	Sensitivity kernel output directory

`forward` is the standard forward simulation mode. `inversion` and `misfit_only` are for Full Waveform Inversion (FWI): the former performs gradient computation using a checkpoint algorithm and the adjoint method, while the latter computes the waveform misfit. Currently, only 2D acoustic waves are supported. The other keys are also for FWI and have not yet been fully debugged.

4.5.4 Output Settings (output)

Key	Type	Required	Default	Description
`output.directory`	string	Optional	`"./"`	Output root directory
`output.summary_file`	string	Optional	`""`	Summary file name (empty string disables output)
`output.log_interval`	int	Optional	100	Log output interval (in time steps)

All output files are written to the directory specified by `output.directory`. `output.summary_file` records a summary of the computation. It is convenient to set this to something like summary.txt. An empty string disables the output. `output.log_interval` specifies how often (in time steps) the computation progress is reported during execution. In parallel computations, MPI communication occurs at each reporting interval, so this should be set to an appropriate value. In GPU computations especially, host/device communication occurs each time, so caution is advised.

4.5.5 Wavefield Output (output.wavefield)

Key	Type	Required	Default	Description
`enabled`	bool	Optional	false	Enable wavefield snapshot output
`interval`	int	Conditional	—	Output interval (in time steps)
`fields`	string array	Optional	`["DISP"]`	Output fields
`formats`	array	Conditional	—	List of output formats

Available values for `fields`: `DISP` (displacement), `VEL` (velocity), `ACC` (acceleration), `PS` (pressure scalar, only for acoustic wave simulations).

Output Formats The following options can be specified for each element of the `formats` array:

Key	Applies to	Default	Description
`type`	All	—	`glvis`, `paraview`, `gmt`
`refinement`	paraview	1	Subdivision level
`data_format`	paraview	`binary32`	`ascii`, `binary` (64-bit), `binary32` (32-bit)
`compression`	paraview	−1	zlib compression level (0–9, −1=default; likely requires MFEM built with zlib, untested)
`resolution`	gmt	—	Output grid size (resolution) `[nx, ny]`
`components`	gmt	`[-1]`	−1=magnitude, 0=x, 1=y, 2=z (multiple values allowed, e.g., [-1, 0, 1, 2])
`cross_sections`	gmt (3D)	—	Cross-section positions

For 3D GMT output, specify cross-section positions with `cross_sections`:

```
cross_sections:
  yz: [2500.0]        # yz cross-section at x=2500m
  xz: [2500.0]        # xz cross-section at y=2500m
  xy: [1000.0]        # xy cross-section at z=1000m
```

Per-side overrides (fluid-solid coupling). When `material.type: coupled` is in use, the fluid and solid sub-domains live on separate `ParSubMesh`es and need separate wavefield files. Add `fluid:` and/or `solid:` sub-blocks under `wavefield:` to override any of `enabled / interval / fields / formats / components` on one side only. Unspecified keys inherit from the parent `wavefield:`. Setting `enabled: false` on a side silences that domain's output while the other side keeps writing snapshots.

```
wavefield:
  enabled: true
  interval: 500
  formats:
    - type: glvis
  fluid:
    fields: [PS]            # only pressure on the fluid side
  solid:
    fields: [DISP, VEL]     # displacement + velocity on the solid side
```

4.5.6 Material Property Output (output.material)

Key	Type	Required	Default	Description
`enabled`	bool	Optional	false	Enable material property output
`fields`	string array	Optional	`["vp"]`	Output fields
`formats`	array	Conditional	—	Formats (same as for wavefield)

Available values for `fields`: `vp`, `vs`, `rho`, `qkappa`, `qmu`. Material property output is written once at the start of the simulation.

For `material.type: coupled` simulations, `fluid:` and `solid:` sub-blocks are accepted under `output.material` with the same override semantics as the wavefield per-side block above (useful for asking only for `vs` on the solid side, etc.).

4.5.7 Example simulation Section Configuration

```
simulation:
  dimension: 3
  order: 4

  time:
    steps: 20000
    dt: 0.00005
    cfl_factor: 0.3

  output:
    directory: "./results"
    summary_file: "summary.txt"
    log_interval: 500

    wavefield:
      enabled: true
      interval: 1000
      fields: ["DISP"]
      formats:
        - type: "paraview"
          refinement: 2
```

```
      data_format: "binary32"
    - type: "gmt"
      resolution: [200, 200]
      components: [-1, 0, 1, 2]
      cross_sections:
        xy: [0.0]
        xz: [2500.0]

material:
  enabled: true
  fields: ["vp", "vs", "rho"]
  formats:
    - type: "paraview"
      refinement: 2
```

4.6 mesh Section

4.6.1 Common Parameters

Key	Type	Required	Default	Description
`type`	string	Required	—	`internal`, `external`, `partitioned`
`max_freq`	real	Optional	—	Maximum frequency for mesh quality evaluation (Hz)
`ppw`	real	Optional	—	Points Per Wavelength
`save`	bool	Optional	false	Save mesh to the results directory

4.6.2 Internal Mesh (type: internal)

Generates a structured grid mesh automatically within SEMSWS.

Key	Type	Required	Default	Description
`origin`	real array	Required	—	Origin coordinates `[x,y]` or `[x,y,z]` (m)
`size`	real array	Required	—	Domain size `[Lx,Ly]` or `[Lx,Ly,Lz]` (m)
`elements`	int array	Required	—	Number of elements `[Nx,Ny]` or `[Nx,Ny,Nz]`
`partition`	string	Optional	`metis`	`metis` or `cartesian`
`partition_grid`	int array	Conditional	—	Grid dimensions for `cartesian` partitioning `[nx,ny,nz]`

`partition: cartesian` provides structured partitioning without requiring METIS. When `partition_grid` is set to, e.g., [4, 4, 2] in 3D, the domain is partitioned into 4, 4, and 2 divisions along the x, y, and z axes, respectively. The number of MPI processes (e.g., –np) must match the product (32 in this case).

4.6.3 External Mesh (type: external)

Key	Type	Required	Default	Description
`file`	string	Required	—	Mesh file path
`format`	string	Optional	Auto-detected	`gmsh`, `mfem`, `exodus`, `vtk`
`partition`	string	Optional	`metis`	MPI partitioning method

When using ExodusII format (CUBIT), MFEM must be built with `MFEM_USE_NETCDF=ON`.

4.6.4 Pre-partitioned Mesh (type: partitioned)

Reads an MFEM-format mesh that has been pre-partitioned using the `partition_mesh` tool. Each MPI rank reads only its own partition file. This is useful for large-scale meshes or when running multiple simulations with the same mesh.

Key	Type	Required	Default	Description
`partitioned.directory`	string	Required	—	Directory containing partition files
`partitioned.nparts`	int	Required	—	Number of partitions (must match the number of MPI processes)

The file naming convention is `mesh.mesh.000000`, `mesh.mesh.000001`, ….

4.6.5 Example mesh Section Configurations

Internal mesh:

```
mesh:
  type: internal
  origin: [0.0, 0.0, 0.0]
  size: [5000.0, 5000.0, 3000.0]
  elements: [50, 50, 30]
  max_freq: 20.0
  ppw: 5.0
  save: true
  partition: cartesian
  partition_grid: [4, 4, 2]
```

External mesh:

```
mesh:
  type: external
  file: "./meshes/model.exo"
  max_freq: 15.0
  ppw: 5.0
```

Pre-partitioned mesh:

```
mesh:
  type: partitioned
  partitioned:
    directory: "./meshes/partitioned"
    nparts: 32
  max_freq: 15.0
  ppw: 5.0
```

4.7 material Section

4.7.1 Material Type and Format

`type` value	Description
`isotropic_acoustic`	Isotropic acoustic medium (vp, rho)
`isotropic_elastic`	Isotropic elastic medium (vp, vs, rho)
`coupled`	Fluid–solid coupled medium; the fluid half is `isotropic_acoustic` and the solid half is `isotropic_elastic`. Nested `fluid:` and `solid:` sub-blocks declare the two physics independently. See §4.7.8.

`format` value	Description
`constant`	Specify homogeneous properties directly in YAML
`grid`	Structured grid ASCII file
`by_attribute`	Text file with properties defined per mesh attribute
`by_attribute_mixed`	YAML file with a mix of constant and grid specifications per attribute
`adios2`	ADIOS2 BP file (for FWI)

`format` is not specified at the top level when `type: coupled` — each of the nested `fluid:` / `solid:` sub-blocks carries its own `format`.

4.7.2 format: constant

Specify material properties directly in YAML.

Key	Type	elastic	acoustic
`vp`	real	Required	Required
`vs`	real	Required	—
`rho`	real	Required	Required

Units are m/s, m/s, and kg/m^3, respectively.

```
material:
  type: isotropic_elastic
  format: constant
  vp: 6000.0
  vs: 3500.0
  rho: 2700.0
```

4.7.3 format: grid

Reads heterogeneous material properties on a structured grid from an ASCII file. Specify the file path with `file`.

File format:

```
Nx Ny [Nz]                      # Grid point counts
dx dy [dz]                      # Grid spacing (m)
x0 y0 [z0]                      # Origin (m)
vp vs rho [Qkappa Qmu]          # Properties at each grid point
vp vs rho [Qkappa Qmu]          # (Nx*Ny[*Nz] rows)
...
```

For elastic media, there are 5 columns (vp, vs, rho, Qkappa, Qmu); for acoustic media, 3 columns (vp, rho, Qkappa). The Q-value columns are required only when attenuation is enabled.

Data is stored in lexicographic order (x-direction varies fastest). The correspondence between indices and coordinates is as follows:

2D The i-th row of data corresponds to grid point (i_x, i_y).

$$i = i_y \times N_x + i_x$$
$$x = x_0 + i_x \times dx, \quad y = y_0 + i_y \times dy$$

That is, N_x points are scanned in the x-direction before advancing one step in the y-direction.

3D The i-th row of data corresponds to grid point (i_x, i_y, i_z).

$$i = i_z \times N_x \times N_y + i_y \times N_x + i_x$$
$$x = x_0 + i_x \times dx, \quad y = y_0 + i_y \times dy, \quad z = z_0 + i_z \times dz$$

That is, the data is scanned in the order $x \to y \to z$. Points outside the grid are clamped to the nearest boundary value.

The origin (x_0, y_0) or (x_0, y_0, z_0) is the minimum coordinate point of the grid and should typically coincide with the mesh `origin`. **In the semsws coordinate system, the y-axis (2D) / z-axis (3D) points upward, so the origin is located at the deepest part of the domain. Consequently, the first row of the file begins with the material properties at the greatest depth.**

4.7.4 format: by_attribute

A text file that defines material properties for each mesh element attribute. Use this when attributes have been assigned using an external meshing tool (CUBIT, etc.).

```
# attr  vp       vs       rho      [Qkappa  Qmu]
1       6000.0   3500.0   2700.0   100.0    100.0
2       3000.0   2000.0   2200.0   50.0     50.0
```

4.7.5 format: by_attribute_mixed

An external YAML file that specifies a mix of constant (homogeneous) and grid (grid file) definitions per attribute.

```
attributes:
  1:
    mode: constant
    vp: 5500.0
    vs: 3175.0
    rho: 2800.0
  2:
    mode: grid
    file: material_layer2.dat
```

4.7.6 format: adios2

Reads material property data on GLL points generated during FWI iterations in ADIOS2 BP format.

Key	Type	Required	Description
`vp_file`	string	Required	BP file path for Vp
`rho_file`	string	Required	BP file path for ρ
`qkappa_file`	string	Optional	BP file path for Q_κ (viscoacoustic only)

4.7.7 Attenuation

Configures attenuation using the generalized Zener model (Standard Linear Solid: SLS).

Key	Type	Required	Default	Description
`attenuation.enabled`	bool	Optional	false	Enable attenuation
`attenuation.f0`	real	Conditional	1.0	Reference frequency (Hz)
`attenuation.n_units`	int	Optional	3	Number of SLS relaxation mechanisms
`attenuation.Qkappa`	real	Conditional	—	Q value for bulk modulus (read only when material.format=constant)
`attenuation.Qmu`	real	Conditional	—	Q value for shear modulus (for elastic constant)

The Q-value fitting band is $[0.1f_0,\ 10f_0]$. For `ascii` and `by_attribute` formats, the Q-value columns in the file are used, so the `Qkappa`/`Qmu` keys are not needed.

4.7.8 type: coupled (Fluid–Solid Coupling)

When a simulation contains both a fluid (acoustic) region and a solid (elastic) region that share an interface, use `material.type: coupled`. The top-level `material:` block then holds two nested sub-blocks `fluid:` and `solid:`, each of which has *its own* `type`, `format`, and material parameters — they are declared independently and may even use different formats (e.g. a constant fluid over a `grid` solid).

Key	Type	Required	Description
`attribute`	int	Required	Parent-mesh element attribute that identifies this subdomain. Must differ between `fluid` and `solid`.
`type`	enum	Required	Must be `isotropic_acoustic` for `fluid`, and `isotropic_elastic` (or `anisotropic_elastic`) for `solid`. A nested `type: coupled` is rejected.
`format`	enum	Required	Same values as for non-coupled materials: `constant`, `grid`, `by_attribute`, `by_attribute_mixed`.
`vp, vs, rho`	real	Conditional	Required for `format: constant` (elastic also needs `vs`).
`file`	path	Conditional	Required for `format: grid / by_attribute / by_attribute_mixed`.
`attenuation`	block	Optional	Optional `attenuation:` sub-block with the same keys as the non-coupled material (§4.7.7). Enabling it on the `fluid:` side activates visco-acoustic; on the `solid:` side, visco-elastic.

The interface boundary attribute between fluid and solid is *auto-detected at run time* by `FluidSolidInterface::Setup`: it is the boundary attribute that exists on exactly one of the two `ParSubMesh` sides but not on the parent's original boundary set. Users do *not* list it in the YAML.

```yaml
# 2D fluid-solid coupling, constant media on both sides,
# with attenuation (Q_kappa = Q_mu = 40) enabled on each domain.
material:
  type: coupled

  fluid:
    attribute: 1
    type: isotropic_acoustic
    format: constant
    vp: 1500.0
    rho: 1000.0
    attenuation:
      enabled: true
      f0: 5
      n_units: 3
      Qkappa: 40

  solid:
    attribute: 2
    type: isotropic_elastic
    format: constant
    vp: 3000.0
    vs: 1732.0
    rho: 2500.0
    attenuation:
      enabled: true
      f0: 5
      n_units: 3
      Qkappa: 40
      Qmu: 40
```

Mesh requirement. The parent mesh must tag every fluid element with the `fluid.attribute` value and every solid element with the `solid.attribute` value. Meshes produced by Cubit or GMSH typically carry these as physical group IDs. The fluid and solid elements *must also be topologically connected* along the interface — duplicate vertices or curves (as left behind by a Cubit `subtract` without a prior `merge vertex / merge curve`) produce a run-time abort explaining the mesh bug.

4.7.9 Model Export

Key	Type	Required	Default	Description
`export_model`	bool	Optional	false	Export the model in ADIOS2 BP format
`export_dir`	string	Optional	`"./model/"`	Export destination directory

4.7.10 Example material Section Configurations

Homogeneous elastic body (with attenuation):

```yaml
material:
  type: isotropic_elastic
  format: constant
  vp: 6000.0
  vs: 3500.0
  rho: 2700.0
```

```
  attenuation:
    enabled: true
    f0: 2.0
    n_units: 3
    Qkappa: 100.0
    Qmu: 50.0
```

Per-attribute definition (when using an external mesh):

```
material:
  type: isotropic_elastic
  format: by_attribute
  file: "materials.txt"
  attenuation:
    enabled: true
    f0: 2.0
    n_units: 3
```

Heterogeneous acoustic medium (grid file):

```
material:
  type: isotropic_acoustic
  format: grid
  file: "velocity_model.dat"
```

4.8 boundary Section

4.8.1 Absorbing Boundary Conditions (absorbing)

Configures absorbing boundary conditions using the Cerjan sponge layer.

Key	Type	Required	Default	Description
`type`	string	Required	—	Currently only `cerjan` is supported
`sides`	string array	Required	—	Sides to apply (empty array `[]` disables)
`thickness`	real	Required	—	Sponge layer thickness (m)
`alpha`	real	Required	—	Damping coefficient (requires tuning; start around 0.4; too strong is counterproductive)

Available side names for `sides`:

Dimension	Available sides
2D	`bottom`, `right`, `top`, `left`
3D	`bottom`, `front`, `right`, `back`, `left`, `top`

Side name ↔ boundary-attribute ID mapping. SEMSWS translates each side name into a fixed, one-based MFEM boundary-attribute integer; the same mapping is applied to `sides` here and to `dirichlet.attributes`. *When you author an external mesh in Cubit, GMSH or another generator, you must stamp the boundary faces with exactly these integers* — otherwise the YAML side names will not resolve to the right physical boundary.

Table 4.1: Side name → MFEM boundary-attribute integer used by SEMSWS. These are the IDs produced by `Mesh::MakeCartesian2D` / `MakeCartesian3D` and must also be applied when tagging faces in external meshers.

Side name	2D attribute	3D attribute
`bottom`	1	1 (min z)
`right`	2	3 (max x)
`top`	3	6 (max z)
`left`	4	5 (min x)
`front`	—	2 (min y)
`back`	—	4 (max y)

`dirichlet.attributes` accepts the side names *and* raw integer attribute IDs interchangeably; the side names are simply aliases for the integers in the table above.

4.8.2 Dirichlet Boundary Conditions (dirichlet)

Key	Type	Required	Default	Description
`attributes`	array	Optional	`[]`	Side names or integer boundary attribute IDs

Side names (`top`, `left`, etc.) and integer attribute IDs can be mixed. Assigning IDs requires an external meshing tool (CUBIT/GMSH, etc.). For internal meshes, only the boundary sides of the mesh domain can be specified using names such as top, bottom, etc. In SEM, unspecified boundaries become natural boundary conditions, so Dirichlet boundaries are required for acoustic wave simulations.

4.8.3 Example boundary Section Configuration

```
# Marine simulation: sea surface as Dirichlet, others as absorbing
boundary:
  absorbing:
    type: cerjan
    sides: [bottom, left, right]
    thickness: 1200.0
    alpha: 0.4
  dirichlet:
    attributes: [top]
```

4.9 sources Section

4.9.1 Source Mode

Key	Type	Required	Default	Description
`mode`	string	Optional	`simultaneous`	`simultaneous`, `sequential`
`file`	string	Optional	—	External source definition YAML file

`simultaneous` fires all sources at the same time. `sequential` processes each source one at a time (loop within the simulation). When `file` is specified, source definitions are loaded from

an external YAML file. This is useful when there are many sources or when source files are automatically generated by scripts in an FWI workflow.

4.9.2 Common Source Parameters (list)

The following are specified for each element of the `sources.list` array:

Key	Type	Required	Default	Description
`id`	int	Required	—	Source ID
`name`	string	Optional	—	Source name (unused in the current version)
`type`	string	Required	—	`force`, `pressure`, `moment_tensor`
`location`	real array	Required	—	Position `[x,y]` or `[x,y,z]` (m)

4.9.3 Source Type: force

Single force for elastic wave simulations.

Key	Type	Required	Description
`direction`	real array	Required	Force direction vector `[x,d]` or `[x,y,z]`

```
- id: 1
  type: force
  location: [500.0, 300.0, 0.0]
  direction: [0.0, 0.0, 1.0]    # vertical direction
  wavelet:
    type: ricker
    frequency: 10.0
    amplitude: 1.0e10
```

4.9.4 Source Type: pressure

Pressure point source for acoustic wave simulations. No additional parameters.

4.9.5 Source Type: moment_tensor

Moment tensor source. Directly specifies the earthquake focal mechanism.
For 3D (6 components): `Mxx`, `Myy`, `Mzz`, `Mxy`, `Mxz`, `Myz`.
For 2D (3 components): `Mxx`, `Myy`, `Mxy`.

```
- id: 1
  type: moment_tensor
  location: [5000.0, 5000.0, 3000.0]
  moment_tensor:
    Mxx: 1.0e18
    Myy: 0.0
    Mzz: 0.0
    Mxy: 0.0
    Mxz: 0.0
    Myz: 0.0
  wavelet:
    type: ricker
    frequency: 5.0
    amplitude: 1.0
```

4.9.6 Wavelet

Key	Type	Required	Default	Description
`type`	string	Optional	`ricker`	`ricker`, `gaussian`, `external`
`frequency`	real	Conditional	—	Dominant frequency (Hz). Required for ricker/gaussian
`amplitude`	real	Optional	1.0	Amplitude scaling
`delay`	real	Optional	0.0	Delay time for peak arrival (s)
`file`	string	Conditional	—	External STF file path (required for external)

For Ricker wavelets, `delay` $\geq 1.2/$`frequency` is recommended.

The external STF file is a two-column ASCII text file (time, amplitude). Note that the time column is not used, so the first row must be aligned with the simulation start time:

```
# time (s)     amplitude
0.0000         0.000000e+00
0.0001         1.234568e-05
...
```

4.9.7 Observed Data (observed) — Inversion Mode

When `simulation.mode` is `inversion` or `misfit_only`, observed data is specified for each source.

Key	Type	Required	Description
`observed.format`	string	Required	`su`, `hdf5`
`observed.files[].file`	string	Required	Data file path
`observed.files[].type`	string	Required	`pressure`, `velocity`, `displacement`
`observed.files[].component`	int	Conditional	0=x, 1=y, 2=z, −1=scalar
`observed.files[].weight_file`	string	Optional	Weight file (SU format)

4.9.8 Example sources Section Configurations

Acoustic waves (multiple sources, simultaneous firing):

```
sources:
  mode: simultaneous
  list:
    - id: 1
      type: pressure
      location: [300.0, 400.0]
      wavelet:
        type: ricker
        frequency: 30.0
        amplitude: 1.0e10
        delay: 0.05
    - id: 2
      type: pressure
      location: [700.0, 400.0]
      wavelet:
        type: ricker
        frequency: 30.0
```

```
        amplitude: 1.0e10
        delay: 0.05
```

Elastic waves (moment tensor, external STF):

```
sources:
  mode: sequential
  list:
    - id: 1
      type: moment_tensor
      location: [5000.0, 5000.0, 3000.0]
      moment_tensor:
        Mxx: 1.0e18
        Myy: -1.0e18
        Mzz: 0.0
        Mxy: 0.0
        Mxz: 0.0
        Myz: 0.0
      wavelet:
        type: external
        frequency: 5.0
        file: "stf_earthquake.txt"
```

The structure of the external file is the same as the `list` within the `sources` section of config.yaml:

```
# config.yaml side
sources:
  mode: simultaneous
  file: "./sources.yaml"

# sources.yaml (external file)
sources:
  - id: 1
    type: pressure
    location: [1800.0, 4990.0]
    wavelet:
      type: ricker
      frequency: 30.0
      amplitude: 1.0e10
      delay: 0.05
  - id: 2
    type: pressure
    location: [2644.0, 4990.0]
    wavelet:
      type: ricker
      frequency: 30.0
      amplitude: 1.0e10
      delay: 0.05
```

4.10 receivers Section

4.10.1 Recording Type and Output Settings

Key	Type	Required	Default	Description
`type`	string/array	Required	—	Default recording type(s) (may be overridden per receiver; see below)
`file`	string	Optional	—	Path to an external YAML file with `list:` / `line:` definitions. See §4.10.4.
`output.formats`	array	Required	—	List of output formats (mapping entries)
`output.filename`	string	Conditional	—	Base file name (required for hdf5/su)

Available values for `type`:

Value	Description	Supported model
`DISP`	Displacement	elastic
`VEL`	Velocity	elastic
`ACC`	Acceleration	elastic
`PS`	Pressure	acoustic

Output formats. `output.formats` is a list of mapping entries, each of which carries a `type:` key selecting one of `ascii` (text, one file per receiver), `hdf5` (single HDF5 file), or `su` (Seismic Unix format). Multiple formats may be requested at once:

```
output:
  formats:
    - type: ascii
    - type: su
  filename: "seismograms"      # required when hdf5 or su is used
```

To inspect the emitted SU files with Seismic Unix, use a build with XDRFLAG disabled.

Per-receiver type override. Each inline receiver entry may override the parent-level `type` with its own `type:` key (scalar or list). This is especially useful with `material.type: coupled`: a fluid-side station can ask for `[PS]` only while a solid-side station asks for `[DISP, VEL, ACC]`, avoiding a "receiver type not found on this side" error from the submesh point locator.

Key	Type	Required	Default	Description
`name`	string	Required	—	Receiver name
`location`	real array	Required	—	Position `[x,y]` or `[x,y,z]` (m)
`type`	string/array	Optional	parent-level `receivers.type`	Per-receiver override of the recording type(s)
`weight`	real	Optional	1.0	Scalar weight (used by FWI observed-data assembly)

4.10.2 Receiver Line (line)

Automatically generates a line of equally spaced receivers.

Key	Type	Required	Default	Description
`line.start`	real array	Required	—	Start position (m)
`line.end`	real array	Required	—	End position (m)
`line.count`	int	Required	—	Number of receivers
`line.prefix`	string	Required	—	Name prefix

Names are formed by appending a 3-digit number to the `prefix` (e.g., `prefix: "REC"` produces `REC001`, `REC002`, …).

4.10.3 Example receivers Section Configurations

List format (individual specification):

```
receivers:
  type: [DISP, VEL]
  output:
    formats:
      - type: ascii
      - type: su
    filename: "seismograms"
  list:
    - name: "R01"
      location: [1000.0, 2500.0, 0.0]
    - name: "R02"
      location: [2000.0, 2500.0, 0.0]
    - name: "R03"
      location: [3000.0, 2500.0, 0.0]
```

Line format (equally spaced, auto-generated):

```
receivers:
  type: [PS]
  output:
    formats:
      - type: ascii
  line:
    start: [100.0, 300.0]
    end: [900.0, 300.0]
    count: 17
    prefix: "REC"
```

Per-receiver type override (typical fluid-solid coupling layout, where R001 lives in the acoustic half and R002/R003 live in the elastic half; each station only records the channels its own physics supports):

```
receivers:
  output:
    formats:
      - type: ascii
  type: [PS, DISP, VEL, ACC]    # union of everything requested below
  list:
    - name: "R001"
      location: [800.0, 200.0]
      type: [PS]
```

```
    - name: "R002"
      location: [800.0, 440.0]
      type: [DISP, VEL, ACC]
    - name: "R003"
      location: [800.0, 680.0]
      type: [DISP, VEL, ACC]
```

4.10.4 External Receiver File (receivers.file)

Large receiver layouts can be factored out of `config.yaml` and loaded from a separate YAML file via the `receivers.file` key. The external file holds top-level `list:` and/or `line:` nodes with exactly the same schema as the inline form; everything else (type, output format, filename) stays in the main config.

```yaml
# config.yaml
receivers:
  type: [PS]
  output:
    formats:
      - type: ascii
  file: "./receivers.yaml"      # overrides any inline list/line below
```

```yaml
# receivers.yaml - paths are resolved from the current working directory
list:
  - name: "REC001"
    location: [100.0, 300.0]
  - name: "REC002"
    location: [200.0, 300.0]
  - name: "REC003"
    location: [300.0, 300.0]

# line: may coexist with list:
# line:
#   start: [0.0, 300.0]
#   end:   [1000.0, 300.0]
#   count: 21
#   prefix: "LINE"
```

When `receivers.file` is set, any inline `list:` / `line:` in the main config is ignored — the external file is authoritative. Loading errors (missing file, malformed YAML) abort the run with a diagnostic pointing at the external path.

4.11 device Section

Key	Type	Required	Default	Description
`type`	string	Optional	`cpu`	`cpu`, `cuda`, `hip`
`seismo_buffer_steps`	int	Optional	0	Receiver waveform buffer size (GPU only)

`type` must match the build configuration. Specifying `cpu` with a GPU build will work but will be extremely slow; a CPU build should be used for CPU execution.

`seismo_buffer_steps` specifies the number of steps for the waveform recording buffer on the GPU. When set to 0, all steps are buffered in GPU memory (fastest but highest memory consumption). When set to $N > 0$, data is transferred to the host every N steps.

4.11.1 Example device Section Configuration

```
device:
  type: cuda
  seismo_buffer_steps: 1000
```

Chapter 5

Tools and Scripts

In addition to the main simulation program (`semsws`), SEMSWS includes preprocessing tools for mesh partitioning, refinement, and quality evaluation, as well as Python scripts for visualizing results. This chapter explains how to use these tools.

5.1 partition_mesh — Mesh Pre-partitioning

`partition_mesh` is a tool that pre-partitions a mesh file for MPI parallel execution (serial run only). Using a pre-partitioned mesh eliminates the overhead of mesh loading and partitioning at simulation startup. This is convenient when running multiple simulations with the same mesh and partition count.

5.1.1 Usage

```
# Partition an external mesh into 64 parts using METIS
./build/src/partition_mesh -n 64 domain.exo partitions/

# Cartesian partitioning (4*4*4 = 64)
./build/src/partition_mesh -n 64 -p cartesian -g 4,4,4 domain.exo
   partitions/

# Generate an internal mesh and partition it
./build/src/partition_mesh -n 64 --internal \
  --size 10000,10000,5000 --elements 100,100,50 partitions/
```

5.1.2 Options

Option	Description
`-n <nparts>`	Number of partitions (required)
`-p <method>`	Partitioning method: `metis` (default) or `cartesian`
`-g <nx,ny,nz>`	Grid dimensions for Cartesian partitioning
`--internal`	Internal mesh generation mode
`--dim <2|3>`	Dimension (for internal mesh, default: 3)
`--origin <x,y,z>`	Origin (for internal mesh)
`--size <lx,ly,lz>`	Domain size (for internal mesh, required)
`--elements <nx,ny,nz>`	Number of elements (for internal mesh, required)

Output files are generated in the specified directory as `mesh.mesh.000000`, `mesh.mesh.000001`, ….

5.1.3 Usage in config.yaml

To use the pre-partitioned mesh in a simulation, configure the `mesh` section of config.yaml as follows:

```
mesh:
  type: partitioned
  partitioned:
    directory: "partitions/"
    nparts: 64
```

`nparts` must match the number of MPI processes at runtime.

5.2 combine_mesh — Combining Partitioned Meshes

`combine_mesh` is a tool that combines meshes partitioned by `partition_mesh` into a single file. It is used for visualization with GLVis and for verifying partitioned meshes.

```
# Combine a 64-partition mesh (run with the same np as the partition count)
mpirun -np 64 ./build/src/combine_mesh partitions/ combined.mesh
```

The number of MPI processes at runtime must match the partition count.

The combined mesh can be viewed with GLVis:

```
glvis -m combined.mesh
```

5.3 evaluate_mesh — Mesh Quality Evaluation

`evaluate_mesh` is a tool that evaluates mesh quality and checks consistency with simulation parameters. It computes statistics such as element size, CFL condition, PPW (Points Per Wavelength), aspect ratio, and non-orthogonality.

5.3.1 Usage

```
# Basic evaluation (console output only)
mpirun -np 4 ./build/src/evaluate_mesh --config config.yaml

# Also output histogram CSV files
mpirun -np 4 ./build/src/evaluate_mesh --config config.yaml \
  --histogram mesh_hist --nbins 50
```

5.3.2 Options

Option	Description
`--config <file>`	Configuration file (required)
`--cfl <factor>`	Override the CFL factor
`--histogram <prefix>`	Output prefix for histogram CSV files
`--nbins <N>`	Number of histogram bins (default: 50)

5.3.3 Output Histograms

When `--histogram` is specified, the following CSV files are generated:

File	Contents
`<prefix>_element_size.csv`	Element size distribution
`<prefix>_dt_cfl.csv`	$\Delta t_{\max}$ distribution from CFL condition
`<prefix>_ppw.csv`	PPW distribution
`<prefix>_aspect_ratio.csv`	Aspect ratio distribution
`<prefix>_non_orthogonality.csv`	Non-orthogonality (degrees) distribution
`<prefix>_condition_number.csv`	Condition number distribution
`<prefix>_jacobian_det.csv`	Jacobian determinant distribution

5.4 refine_mesh — Uniform Mesh Refinement

`refine_mesh` is a tool that uniformly refines a mesh based on material properties. It automatically subdivides elements in low-velocity regions (where wavelengths are short) to generate a mesh that satisfies the PPW requirement.

The refinement criterion is determined by the following formula. First, the required element size h_{required} is computed:

$$h_{\text{required}} = \frac{v_{\min} \times N_{\text{GLL}}}{f_{\max} \times \text{PPW}}$$

where $v_{\min}$ is the minimum wave velocity across all elements, $N_{\text{GLL}} =$ `order` $+ 1$, and $f_{\max}$ and PPW are specified by `mesh.max_freq` and `mesh.ppw` in the configuration.

Next, the number of refinement levels n is determined from the current element size h_{current}:

$$n = \left\lceil \log_2 \frac{h_{\text{current}}}{h_{\text{required}}} \right\rceil$$

If $h_{\text{current}} \leq h_{\text{required}}$, no refinement is needed ($n = 0$). The maximum n across all elements becomes the refinement level for the entire mesh.

Note that the entire mesh is refined uniformly. Each refinement level bisects every element, so the element count increases by a factor of 2^{2n} in 2D and 2^{3n} in 3D. Therefore, some compromise on metrics such as PPW may be necessary.

The primary use case is generating meshes that satisfy the PPW criterion. A second use case involves large-scale HPC computations with large meshes. **In SEMSWS, even in parallel computations (except when using `mesh.type.partitioned` in config.yaml), rank 0 reads the entire mesh and then partitions and distributes it via MPI_SEND. This can cause out-of-memory issues when handling large meshes. Furthermore, meshes pre-partitioned by external tools such as CUBIT cannot be read. The `partition_mesh` pre-partitioning tool also initially reads the entire mesh on a single rank.** In such cases, a coarse mesh that captures the basic geometry (topography, bathymetry, etc.) can be created with CUBIT or GMSH, and then refined using this tool. As described below, the refinement process reads the input mesh on rank 0, partitions it, refines it in parallel on each rank, and outputs the partitioned mesh. The output mesh can be used by setting `mesh.type: partitioned` in the configuration, allowing each rank to read only its own partition.

5.4.1 Mesh Loading and Output Behavior

`refine_mesh` internally calls `LoadParMesh()` to load the mesh. The loading path depends on the `mesh.type` setting in the configuration:

Non-partitioned mesh (type: internal / external) Only rank 0 reads the serial mesh (or generates it for internal meshes), partitions it into N parts using `MeshPartitioner`, and distributes the parts to each rank via `MPI_Send`. N is determined by the number of processes specified with `mpirun -np N`.

Pre-partitioned mesh (type: partitioned) Each rank independently reads its own partition file (`mesh.mesh.NNNNNN`). The number of MPI processes at runtime must match `nparts`.

Refinement and output In both cases, the refinement itself is performed in parallel via `ParMesh::UniformRefinement()`, with no difference in behavior. The only difference is in the loading path. After refinement, there are two output options:

- `--output-dir`: Each rank writes its own partition file using `ParPrint()` (N files). Use with `type: partitioned` in simulation configuration
- `--save-as-one`: Rank 0 combines all partitions into a single file using `SaveAsOne()`. Use for GLVis visualization or with `type: external` in simulation configuration

5.4.2 Usage

```
# Run with 8 processes (MPI required)
mpirun -np 8 ./build/src/refine_mesh --config config.yaml --output-dir MESH
    /

# Also save a combined mesh for visualization
mpirun -np 8 ./build/src/refine_mesh --config config.yaml \
  --output-dir MESH/ --save-as-one refined.mesh
```

5.4.3 Options

Option	Description
`--config <file>`	Configuration file (required)
`--output-dir <dir>`	Output directory for partitioned mesh (required)
`-n <max_ref>`	Override the maximum refinement level
`--save-as-one <file>`	Also save a combined mesh (for GLVis visualization)

The output is in the same format as partition_mesh (`mesh.mesh.NNNNNN`) and can be used directly as `type: partitioned` in config.yaml.

5.5 Workflow Example: Mesh Evaluation and Refinement

This section demonstrates the complete workflow—from mesh evaluation through refinement to re-evaluation—using the two-layer viscoelastic model in `examples/visco_elastic_3d_layered/`.

5.5.1 Problem Setup

A two-layer viscoelastic medium is defined in a 10 km × 10 km × 10 km 3D domain. The upper layer ($z > -4000$ m) has low velocity ($V_p = 3000$ m/s, $Q_\kappa = Q_\mu = 50$), and the lower layer ($z \leq -4000$ m) has high velocity ($V_p = 6000$ m/s, $Q_\kappa = Q_\mu = 100$).

The mesh is generated using CUBIT (the free Coreform version is sufficient). Running the included journal file `mesh_layered_3d.jou` generates a mesh in ExodusII format:

```
# Execute the following in the CUBIT GUI CONSOLE
playback 'mesh_layered_3d.jou'
```

Contents of the material property file `materials_visco.txt`:

```
# attribute  vp       vs       rho      qkappa   qmu
1            6000.0   3500.0   2700.0   100.0    100.0
2            3000.0   2000.0   2200.0   50.0     50.0
```

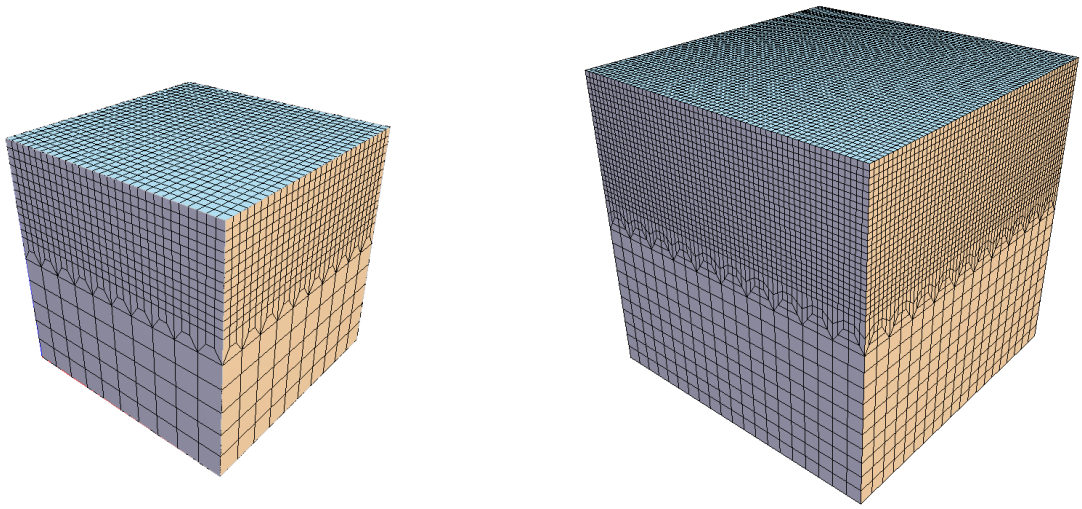

Figure 5.1: Mesh before refinement (left) and after refinement (right). Left: Initial mesh generated by CUBIT (12,600 elements). Upper layer uses 500 m elements, lower layer uses 1,000 m elements. Right: After one uniform refinement with `refine_mesh` (100,800 elements). All elements are subdivided by a factor of $2^3 = 8$.

5.5.2 Step 1: Initial Mesh Evaluation

First, evaluate the quality of the initial mesh:

```
./build/src/evaluate_mesh --config config_visco.yaml --histogram mesh
```

Example output:

```
Mesh Evaluation Report
======================
Mesh: 12600 elements (global), dimension 3
Order: 4, CFL factor: 0.3, f_max: 5.6 Hz, PPW target: 5

Per-element statistics:
                            min          max         mean          std
  h [m]             3.333e+02   1.000e+03   3.858e+02   1.561e+02
  dt_CFL [s]        2.141e-03   8.562e-03   5.394e-03   1.227e-03
  PPW               3.163e+00   9.490e+00   5.436e+00   7.247e-01
  Aspect ratio      1.000e+00   8.492e+00   1.331e+00   1.191e+00
  Non-orth [deg]    1.144e-04   5.631e+01   4.545e+00   1.431e+01
  Cond kappa(J)     1.000e+00   3.400e+00   1.097e+00   3.694e-01
  det(J)            2.778e+07   1.000e+09   7.540e+07   1.883e+08

CFL summary:
  dt_max (global): 2.1406e-03 s
```

```
PPW summary:
  Worst PPW: 3.16 at centroid (9500.00, 4500.00, -6500.00)
  PPW < 5.0: 1300 / 12600 (10.32%)

Per-attribute breakdown:
  Attr 1: 1800 elems, Vp=6050, Vs=3543, PPW_min=3.2, dt_min=2.14e-03
  Attr 2: 10800 elems, Vp=3066, Vs=2049, PPW_min=5.5, dt_min=5.63e-03
```

The minimum PPW is 3.16, which is below the target of 5. In Attr 1 (lower layer, high-velocity zone), 1300 elements (10.3%) have insufficient PPW.

5.5.3 Step 2: Uniform Refinement

Refine the mesh using `refine_mesh`. Here we run with 4 processes, though the included `run_visco.sh` uses 64 processes:

```
mpirun -np 4 ./build/src/refine_mesh \
  --config config_visco.yaml \
  --output-dir mesh/
```

Example output:

```
Loading mesh...
  Elements (global): 12600

Loading material...
  V_min (global): 2049.22
  V_max (global): 6050.01

Refinement levels needed: 1
  Refinement 1: 100800 elements (global)

Refined mesh:
  Elements (global): 100800
  h range:           [84.12, 500.03]
  PPW (worst):       6.33 (target: 5)
  CFL dt_max:        0.00107 s (CFL=0.3)

Saving partitioned mesh to: mesh/
  4 partition files (mesh.mesh.NNNNNN)
```

One refinement level increased the element count from 12,600 to $8\times = 100{,}800$, and the worst PPW improved from 3.16 to 6.33 (Figure 5.1).

5.5.4 Step 3: Re-evaluation After Refinement

Re-evaluate the refined mesh to confirm that the PPW criterion is satisfied. `config_visco_sim.yaml` references the refined mesh via `mesh.type: partitioned`:

```
mpirun -np 4 ./build/src/evaluate_mesh --config config_visco_sim.yaml
```

Example output:

```
Mesh Evaluation Report
======================
Mesh: 100800 elements (global), dimension 3
Order: 4, CFL factor: 0.3, f_max: 5.6 Hz, PPW target: 5

Per-element statistics:
                          min         max        mean         std
```

```
  h [m]             1.667e+02   5.000e+02   1.938e+02   8.145e+01
  dt_CFL [s]        1.070e-03   4.281e-03   2.718e-03   5.622e-04
  PPW               6.326e+00   1.898e+01   1.087e+01   1.563e+00
  Aspect ratio      1.000e+00   8.495e+00   1.230e+00   8.541e-01
  Non-orth [deg]    4.730e-04   5.635e+01   3.923e+00   1.252e+01

PPW summary:
  Worst PPW: 6.33 at centroid (250.00, 9250.00, -9250.00)
  PPW < 5.0: 0 / 100800 (0.00%)

Per-attribute breakdown:
  Attr 1: 14400 elems, Vp=6050, Vs=3543, PPW_min=6.3, dt_min=1.07e-03
  Attr 2: 86400 elems, Vp=3066, Vs=2049, PPW_min=11.0, dt_min=2.82e-03
```

The percentage of elements with insufficient PPW is now 0%, and all elements satisfy PPW ≥ 5.

5.5.5 Step 4: Running the Simulation

Run the simulation with the refined mesh:

```
mpirun -np 64 ./build/src/semsws \
  --config config_visco_sim.yaml
```

5.6 Visualization Scripts

Python visualization scripts are provided in `scripts/plot/`. All of them require `matplotlib` and `numpy`.

5.6.1 plot_wavefield_2d.py — 2D Wavefield Visualization

Visualizes 2D wavefield snapshots and material property distributions output in GMT format using matplotlib.

```
python3 scripts/plot/plot_wavefield_2d.py --results-dir ./results
```

The following files are generated in `results/figures/`:

- `wavefield_snapshots.png` — Wavefield snapshots (all time steps)
- `material.png` — Material property distribution

See Figure 3.1 in Chapter 3 for example output.

5.6.2 plot_wavefield_3d.py — 3D Cross-section Wavefield Visualization

Visualizes 3D cross-sections (cross_sections) in GMT format using matplotlib.

```
python3 scripts/plot/plot_wavefield_3d.py --results-dir ./results
```

Snapshots are generated for each cross-section (yz, xz, xy) specified in the configuration. Figure 5.2 shows an example xz cross-section ($y = 5000\,\mathrm{m}$) output for the two-layer viscoelastic model.

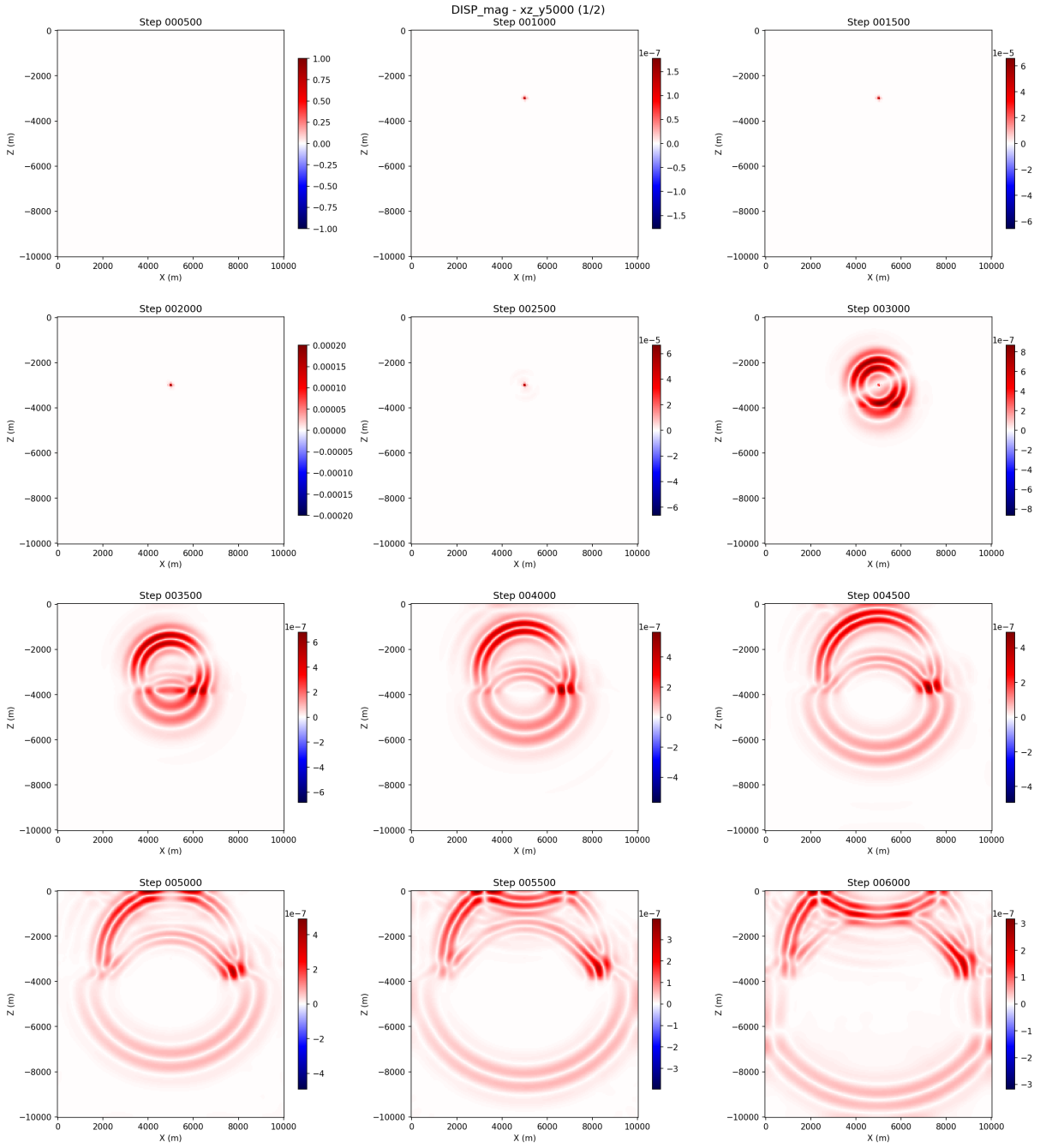

Figure 5.2: 3D cross-section snapshot from `plot_wavefield_3d.py` (xz cross-section, $y = 5000\,\mathrm{m}$). Displacement magnitude for the two-layer viscoelastic model. Output at 500-step intervals.

5.6.3 plot_3d_slices.py — PyVista 3D Visualization

Uses PyVista to create a visualization with three cross-sections arranged in 3D space. The `pyvista` package is additionally required.

```
python3 scripts/plot/plot_3d_slices.py --results-dir ./results \
  --type wavefield --step 6000
```

Option	Description
`--results-dir`	Results directory
`--type`	`wavefield` or `material`
`--step`	Time step number to display (for wavefield)

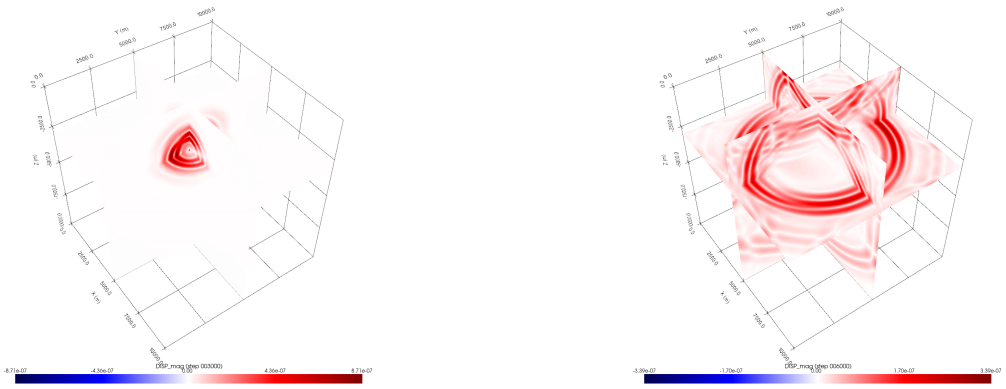

Figure 5.3: PyVista 3D visualization from `plot_3d_slices.py`. yz, xz, and xy cross-sections arranged in 3D space. Left: Step 3000. Right: Step 6000. Displacement magnitude for the two-layer viscoelastic model.

5.6.4 plot_mesh_quality.py — Mesh Quality Histograms

Generates histogram plots from the CSV files output by `evaluate_mesh`.

```
# Output CSV files with evaluate_mesh
mpirun -np 4 ./build/src/evaluate_mesh --config config.yaml \
  --histogram mesh_hist

# Generate histogram plots
python3 scripts/plot/plot_mesh_quality.py mesh_hist
python3 scripts/plot/plot_mesh_quality.py mesh_hist --output quality.png
```

Histograms of element size, CFL $\Delta t_{\max}$, PPW, aspect ratio, non-orthogonality, condition number, and Jacobian determinant are combined into a single figure.

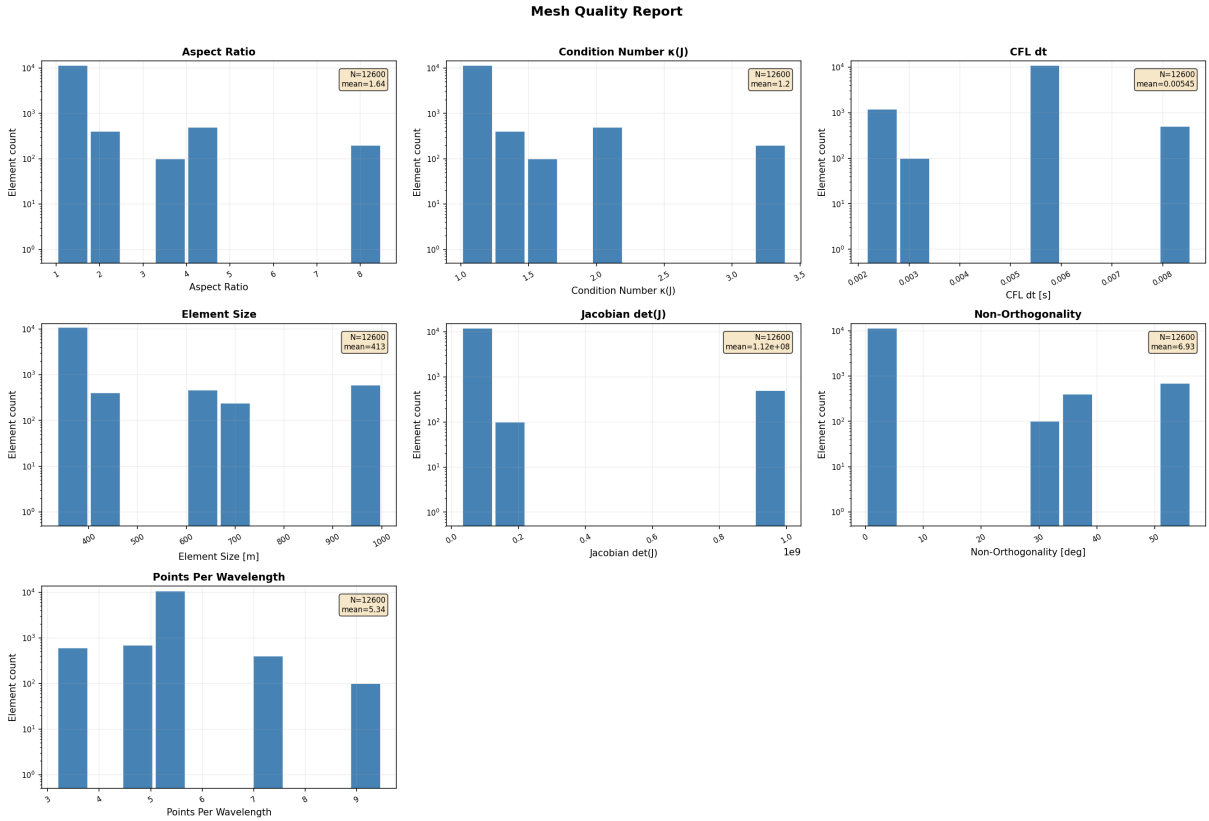

Figure 5.4: Mesh quality histograms (initial mesh before refinement). Output from `evaluate_mesh` visualized with `plot_mesh_quality.py`.

Chapter 6

Application Examples

The `examples/` directory contains sample cases that demonstrate the various features of SEM-SWS. This chapter introduces the main examples, excluding those related to FWI. Each example includes a README and execution scripts, and can be run by following the provided instructions.

6.1 visualization — Visualization Demo

`examples/visualization/` contains demos for testing all output formats (GLVis, ParaView, GMT) with 2D/3D acoustic and elastic wave simulations.

Directory	Model	Description
`2D/acoustic/`	2D acoustic	Used in Chapter 3
`2D/elastic/`	2D elastic	Force source
`3D/acoustic/`	3D acoustic	3D cross-section output (cross_sections)
`3D/elastic/`	3D elastic	Moment tensor source

Each example can be run with `run.sh`:

```
cd examples/visualization/3D/elastic
bash run.sh 4    # argument is the number of MPI processes
```

`run.sh` automatically performs visualization using `plot_wavefield_2d.py` (2D) or `plot_wavefield_3d.py` (3D) after running the simulation.

In the 3D examples, cross-section positions are specified via `cross_sections` in the configuration. Figures 5.2 and 5.3 in Chapter 5 were generated with a setup similar to the 3D elastic wave example.

6.2 elastic_analytic — Analytical Solution Benchmark

`examples/elastic_analytic/` is a benchmark that compares numerical results against the Aki & Richards (2002) analytical solution for a double-couple source in a homogeneous isotropic elastic medium.

6.2.1 Problem Setup

Parameter	Value
Domain	44 km × 44 km × 34 km
Mesh	Internally generated, 120 × 120 × 100 elements
Polynomial order	4
Material properties	$V_p = 2800\,\mathrm{m/s}$, $V_s = 1500\,\mathrm{m/s}$, $\rho = 2300\,\mathrm{kg/m^3}$
Source	Double-couple (M_{rp} component), at domain center
Receivers	6 stations (Z1–Z6), placed at varying distances from the source

6.2.2 How to Run

```
cd examples/elastic_analytic
bash run_benchmark.sh 4    # argument is the number of MPI processes
```

This script automatically executes the following three steps:

1. Generate the analytical solution with `generate_analytical.py` (`ref/*.semd`)
2. Run the simulation with `semsws`
3. Compute and display the normalized L2 error for each trace (6 stations × 3 components = 18 traces)

6.2.3 Visualizing the Results

```
python3 plot_comparison.py
```

A waveform comparison plot for 3 stations × 3 components is generated. The analytical solution (black solid line), numerical solution (red dashed line), and residual ×10 (blue line) are overlaid.

6.3 visco_elastic_3d_layered — 3D Viscoelastic Two-layer Model

`examples/visco_elastic_3d_layered/` is a 3D two-layer model example that uses an external mesh generated by CUBIT. The mesh evaluation and refinement workflow is described in detail in Section 5.5 of Chapter 5.

6.3.1 Included Configurations

File	Description
`config_elastic.yaml`	Elastic (no attenuation), direct external mesh loading
`config_elastic_sim.yaml`	Elastic, refined partitioned mesh
`config_visco.yaml`	Viscoelastic (with attenuation), direct external mesh loading
`config_visco_sim.yaml`	Viscoelastic, refined partitioned mesh
`mesh_layered_3d.jou`	CUBIT journal file
`materials_elastic.txt`	Elastic properties (Q=9999, effectively no attenuation)
`materials_visco.txt`	Viscoelastic properties ($Q_\kappa = Q_\mu = 50$–100)

6.3.2 How to Run

```
cd examples/visco_elastic_3d_layered

# 1. Generate mesh with CUBIT
coreform_cubit -nographics -input mesh_layered_3d.jou

# 2. Refine mesh (64 processes)
mpirun -np 64 ../../build/src/refine_mesh \
  --config config_visco.yaml --output-dir mesh/

# 3. Run simulation
mpirun -np 64 ../../build/src/semsws \
  --config config_visco_sim.yaml
```

Alternatively, run everything at once with the included script:

```
bash run_visco.sh
```

A script for comparing elastic and viscoelastic waveforms is also provided:

```
python3 plot_comparison.py
```

6.4 fun

`examples/fun/` contains examples using unstructured meshes generated by the HOHQMesh library. These were created casually during a break from coding, so there may be issues with the configurations. HOHQMesh is an open-source high-order hexahedral/tetrahedral mesh generation tool that can create meshes of complex shapes using text-based input files.

These examples serve as references for using mesh sources other than CUBIT/GMSH.

6.4.1 ICM — 2D Elastic Waves

`examples/fun/icm/` is a 2D elastic wave simulation on an "ICM"-shaped mesh. A source is placed at the dot of the letter "i". After running a forward simulation to compute the wave propagation, the snapshots are arranged in reverse order to generate an animation of the waves converging toward the dot of "i".

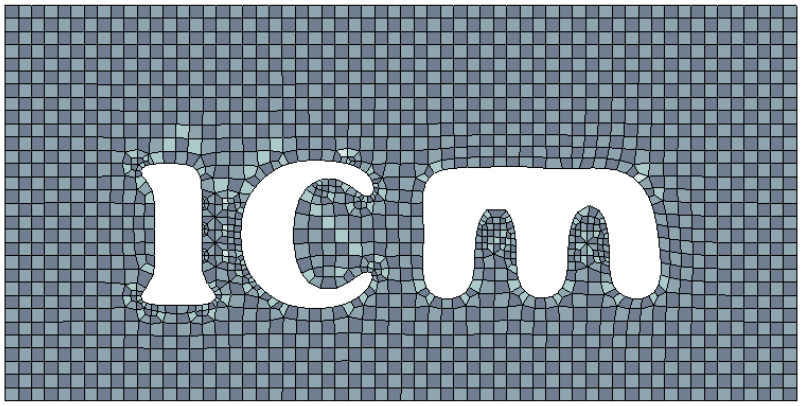

Figure 6.1: “ICM” letter-shaped mesh generated by HOHQMesh.

```
cd examples/fun/icm
bash run.sh
```

File	Description
`icm.control`	HOHQMesh control file
`generate_mesh.py`	Converts HOHQMesh output to GMSH format
`config.yaml`	Simulation configuration (2D elastic, order 8)
`make_animation.py`	Generates animation from snapshots

Demo video (reverse playback animation): https://youtu.be/Qts2n_2wtfI

6.4.2 JAMSTEC — 2D Acoustic Waves

`examples/fun/jamstec/` is a 2D acoustic wave simulation on a “JAMSTEC”-shaped mesh. Similar to ICM, a source is placed at the top of the letter “J”, and a reverse-playback animation of waves converging is generated.

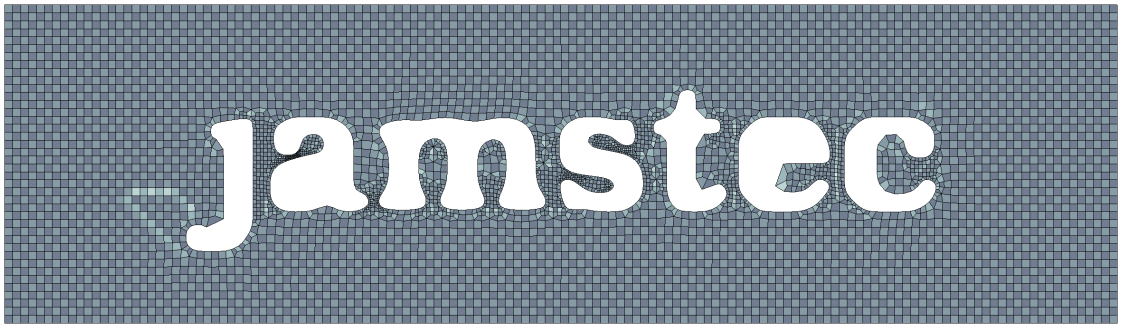

Figure 6.2: “JAMSTEC” letter-shaped mesh generated by HOHQMesh.

```
cd examples/fun/jamstec
bash run.sh
```

Demo video (reverse playback animation): https://youtu.be/_yijXLc3F6U

In these examples, meshes generated by HOHQMesh (`.msh`) are loaded using `mesh.type: external`. The same approach can be used for simulations with meshes of arbitrary shapes.

Chapter 7

Performance

This chapter presents the parallel performance of SEMSWS on the Earth Simulator (ES, JAMSTEC) and the LUMI supercomputer (CSC, Finland). These results are cited from the associated paper.

7.1 Test Environment

Table 7.1: Systems used for testing

System	Partition	Configuration
ES (JAMSTEC)	CPU	AMD EPYC 7742 (128 cores/node), 720 nodes
ES (JAMSTEC)	GPU	NVIDIA A100 (8 GPUs/node), 8 nodes
LUMI (CSC)	GPU	AMD MI250X (4 modules = 8 GCDs/node), 2978 nodes

All tests solve the 3D isotropic elastic wave equation with 4th-order polynomials (5 GLL points) for 1000 time steps. The mesh is a structured hexahedral mesh with Cartesian partitioning, and GPU-aware MPI is used for GPU runs.

7.2 Strong Scaling

In the strong scaling tests, the problem size is fixed at 5.24 million elements (337.3 million mesh nodes) and the number of processes is increased.

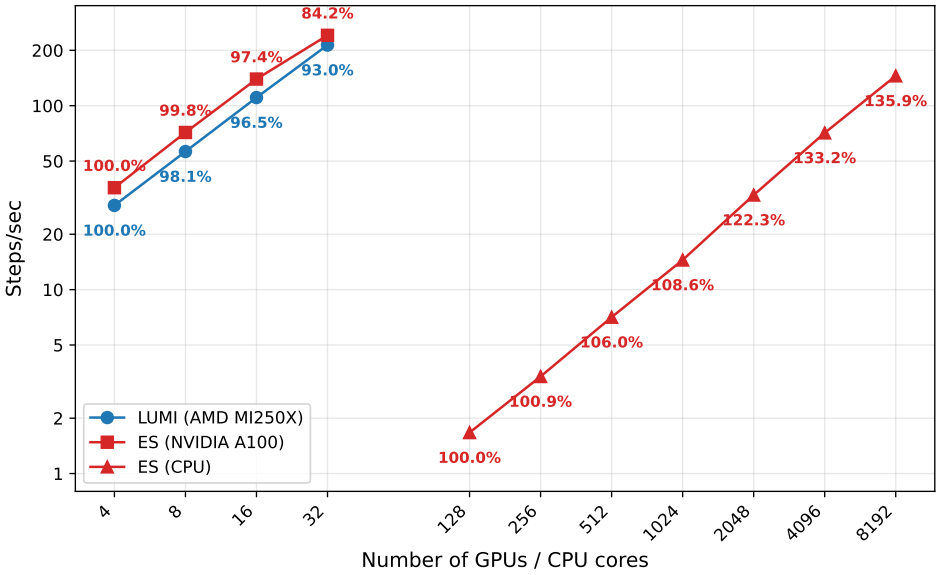

Figure 7.1: Strong scaling results. ES CPU nodes (AMD EPYC 7742), ES GPU nodes (NVIDIA A100), LUMI GPU nodes (AMD MI250X). Numbers next to each symbol indicate parallel efficiency (%). Baseline is CPU: 128 cores, GPU: 4 units.

On ES CPU, super-linear scaling (135.9% parallel efficiency) was observed up to 8192 cores. This is attributed to improved cache efficiency as the per-core problem size decreases. On GPUs, parallel efficiency of 84.2% (ES) and 93.0% (LUMI) was maintained up to 32 GPUs.

7.3 Weak Scaling

In the weak scaling tests, the per-process problem size is fixed at 15,210 elements per CPU and 1,600,000 elements per GPU, with the total problem size increasing proportionally to the number of processes.

Table 7.2: Weak scaling test problem sizes

CPU (ES, 15,210 elements/core)				GPU (ES & LUMI, 1,600,000 elements/GPU)			
nCPU	Elements	Mesh nodes	DoFs	nGPU	Elements	Mesh nodes	DoFs
128	1.95M	125.4M	376.2M	4	6.40M	412.7M	1.24B
512	7.79M	500.8M	1.50B	8	12.8M	825.1M	2.48B
2048	31.2M	2.00B	6.00B	16	25.6M	1.65B	4.95B
8192	124.6M	8.00B	24.0B	32	51.2M	3.30B	9.90B
				GPU (LUMI only)			
				320	512.0M	32.8B	98.4B
				640	1.02B	65.6B	196.8B
				1280	2.05B	131.2B	393.6B
				2560	4.10B	262.4B	787.2B
				5120	8.19B	524.8B	1.57T
				8192	13.1B	839.6B	2.52T

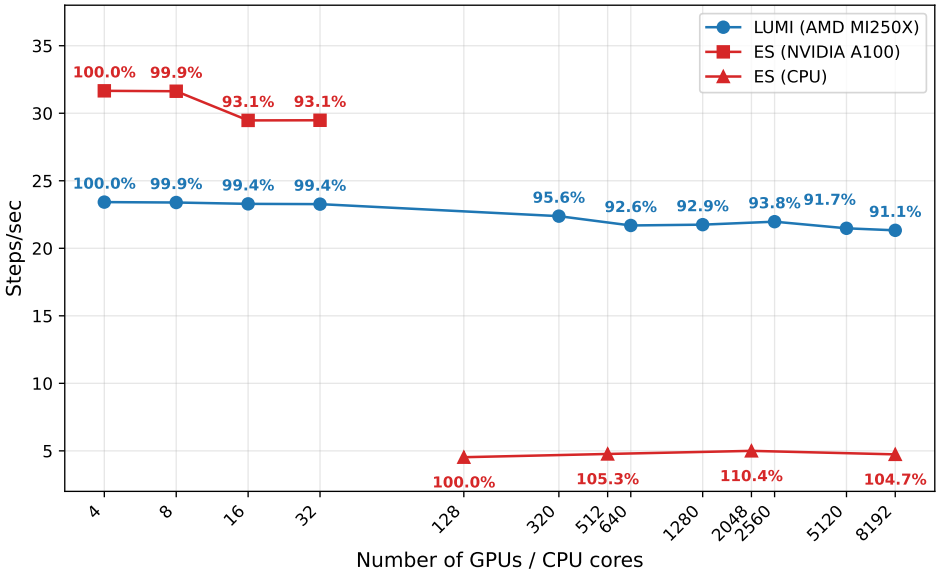

Figure 7.2: Weak scaling results. ES CPU nodes, ES GPU nodes, LUMI GPU nodes. Baseline is CPU: 128 cores, GPU: 4 units. Note that the per-process problem size differs between CPU and GPU.

On CPU, parallel efficiency above 100% was maintained up to 8192 cores. For GPU computations, parallel efficiency exceeded 93% up to 32 NVIDIA A100s on ES and exceeded 91% up to 8192 AMD MI250Xs on LUMI. At the largest configuration (LUMI, 8192 GPUs), the computation involved 839.6 billion mesh nodes (2.52 trillion DoFs).

7.4 GPU vs CPU

Using the strong scaling results on ES, the performance of 1 GPU node (8 NVIDIA A100s) is compared against N CPU nodes (128 cores/node).

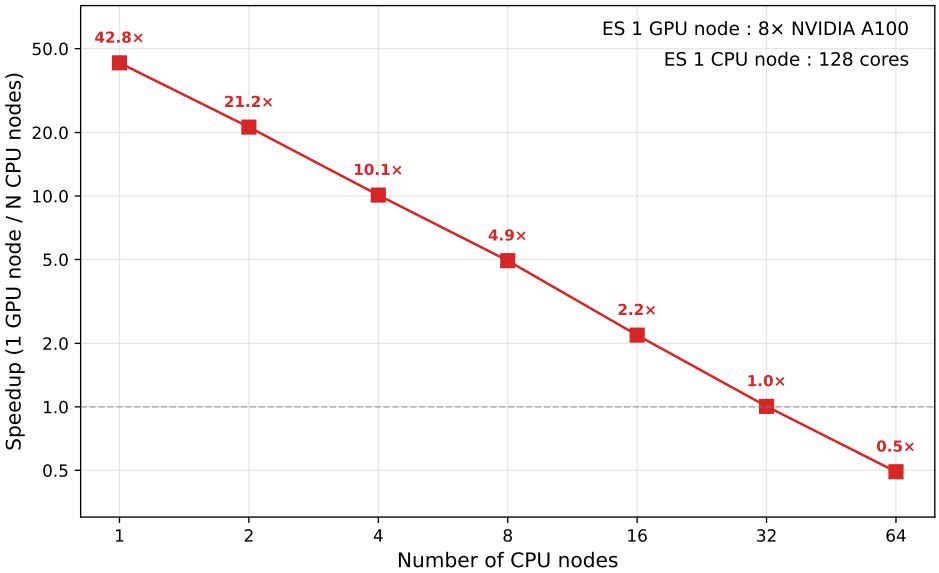

Figure 7.3: Speedup of 1 GPU node (8 NVIDIA A100s) over N CPU nodes (128 cores/node). Problem size is 5.24 million elements.

One GPU node achieved a 42.8x speedup over one CPU node. The performance of one GPU node is equivalent to that of 32 CPU nodes (4,096 cores).

Development Team

- Kota Mukumoto — Barcelona Center for Subsurface Imaging, Institut de Ciències del Mar (CSIC), Barcelona, Spain
- Yann Capdeville — Nantes Université, Univ Angers, Le Mans Université, CNRS, Laboratoire de Planétologie et Géosciences, Nantes, France
- Kazuya Shiraishi — Japan Agency for Marine-Earth Science and Technology (JAMSTEC), Japan
- Ryoichiro Agata — Japan Agency for Marine-Earth Science and Technology (JAMSTEC), Japan
- Gou Fujie — Japan Agency for Marine-Earth Science and Technology (JAMSTEC), Japan
- Thomas Bodin — Barcelona Center for Subsurface Imaging, Institut de Ciències del Mar (CSIC), Barcelona, Spain